コンピュータサイエンス第一 第二回 予習資料 (基本的な制御構文)

この回で学ぶこと

- 条件によって処理を切り替える (if / elif / else)
- 同じ処理を繰り返す (for / while)
- 比較演算子と論理演算子
- インデント (字下げ) が Python では「意味」を持つこと

前回までの復習

- 変数は `a = 1` のように**代入**して作る (= は等号ではなく代入演算子)。
- 「+」は、整数同士なら足し算、文字列同士なら結合になる。
- 今回は、プログラムが**上から順に実行されるだけではない**ことを学びます。

1. 条件分岐 (if 文)

1-1. 基本の考え方

「もし雨が降っていたら傘を持つ、そうでなければ持たない」のように、条件によってやることを変えたいときに使います。

1-2. 書き方

```
x = 7
if x > 5:
    print("5 より大きい")
elif x == 5:
    print(" ちょうど 5")
else:
    print("5 より小さい")
```

ポイント

`if` の条件が成り立つ (True) ときだけ、字下げされた部分が実行されます。条件の後ろの `:`(コロン) を忘れないこと。

細かい解説

`x > 5` のような式は「条件式」と呼ばれ、評価すると `True`(成り立つ)か `False`(成り立たない)のどちらかの値になります。試しに `print(7 > 5)` と書くと `True` と表示されます。`if` / `elif` / `else` は上から順に調べられ、**最初に成り立ったものだけ**が実行されて残りは無視されます。上の例では `x = 7` なので最初の `x > 5` が成り立ち、`elif` 以降は調べられません。`elif` は何個でも書けますし、`else` は必要なければ省略できます。字下げされた範囲(ブロック)が「条件が成り立ったときに実行する処理のまとまり」です。ブロックの中には何行でも書けます。字下げをやめて元の位置に戻ると、そこからは条件と関係なく常に実行されます。

注.

- 「等しい」は `==`(イコール２個)です。`=` １個は代入なので別物です。
- 字下げは半角スペース４つが標準です。字下げが揃っていないとエラーになります。

1-3. 比較演算子と論理演算子

記号	意味	例
`==`	等しい	`a == 3`
`!=`	等しくない	`a != 3`
`< >`	小さい・大きい	`a < 3`
`<= >=`	以下・以上	`a >= 3`
`and`	かつ	`a > 1 and a < 5`
`or`	または	`a < 1 or a > 5`
`not`	否定	`not a == 3`

細かい解説

第１回で学んだ型の違いは、比較でも効いてきます。`"1" == 1` は文字列と整数なので `False` です(見た目は同じ「1」でも別物)。また、`a > 1 and a < 5` は Python では `1 < a < 5` と短く書くこともできます。`and` は両方とも成り立つときだけ、`or` はどちらか一方でも成り立てば `True` になります。

2. 繰り返し (for 文)

2-1. 基本の考え方

「同じことを 10 回やりたい」ときに、10 行書く代わりに繰り返しの仕組みを使います。

2-2. 書き方

```
for i in range(3):
    print(i)
```

出力は次のようになります。

```
0
1
2
```

ポイント

`range(3)` は 0, 1, 2 という 3 つの数を順に作ります。**0 から始まり、3 は含まない**ので注意。

細かい解説

`for i in range(3):` の `i` は、繰り返しのたびに 0, 1, 2 と値が入れ替わる**ただの変数**です。名前は自由に付けられます (慣習的に `i` がよく使われます)。`range` には引数を 2 つ、3 つ渡すこともできます。

```
range(1, 4)     # 1, 2, 3  (開始, 終了) 終了は含まない
range(0, 10, 3) # 0, 3, 6, 9 (開始, 終了, 増加量)
```

また、`range` 以外にも、文字列を渡すと 1 文字ずつ取り出せます。

```
for ch in "abc":
    print(ch)   # a b c が 1 行ずつ表示される
```

3. 条件つきの繰り返し (while 文)

```
n = 0
while n < 3:
    print(n)
    n = n + 1
```

ポイント

条件が成り立つ間、繰り返します。`n = n + 1` のように条件が変わる処理がないと、**永遠に終わらない (無限ループ)** ので注意しましょう。

細かい解説

回数が決まっているときは `for`、「条件が変わるまで続けたい」ときは `while` が向いています。万一無限ループになってしまったら、ターミナルで `Ctrl + C` を押すと強制的に止められます。慌てなくて大丈夫です。上の例では、`n` が 0 → 1 → 2 → 3 と増え、`n < 3` が成り立たなくなった時点で終了します。そのため、表示されるのは 0, 1, 2 です。

4. break と continue

- `break`: 繰り返しを途中で終了する
- `continue`: 今回の周だけ飛ばして次の周へ進む

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

細かい解説

このコードは 0, 1, 2 と表示したところで、`i` が 3 になって `break` が実行され、ループ全体が終わります。`break` の代わりに `continue` を使うと、3 のときだけ表示を飛ばして 0, 1, 2, 4 と表示されます。

5. 予想してみよう

例題 1

```
a = 4
if a % 2 == 0:
    print(" 偶数")
else:
    print(" 奇数")
```

答え: **偶数** です。`%` は割り算の余りを求める演算子で、4 を 2 で割った余りは 0 だからです。

細かい解説

`%` は「剰余演算子」と呼ばれます。`7 % 2` は 1、`10 % 3` も 1 です。「2 で割った余りが 0 なら偶数」は、条件分岐でよく使うパターンなので覚えておくと便利です。

例題 2

```
total = 0
for i in range(1, 4):
    total = total + i
print(total)
```

答え:**6** です。`i` は 1, 2, 3 と変化し、`1 + 2 + 3` が足されます。

用語まとめ

用語	意味
制御構文	処理の流れを変える仕組み
条件式	True か False になる式
インデント	行頭の字下げ。ブロックの範囲を表す
ループ	繰り返し
無限ループ	終わらない繰り返し